



Laboratório 3: Implementação de algoritmos comuns de Machine Learning

1 Introdução

1.1 Sobre Este Laboratório

Este laboratório apresenta algoritmos comuns de Machine Learning para ajudar você a melhor entender suas funções e aplicabilidades. Especificamente, explicará como criar um algoritmo de regressão linear do zero e implementar a árvore de decisão e o agrupamento K-means com base no `scikit-learn`.

O ambiente do laboratório é o Python 3.7. Para obter mais detalhes sobre como configurar o ambiente, confira o Guia de Configuração do Ambiente. Os conjuntos de dados usados neste laboratório podem ser obtidos em:

<https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V3.5/chapter2/ML.zip>

1.2 Objetivos

- Criar um algoritmo de regressão linear do zero.
- Dominar o uso de algoritmos de regressão e classificação.
- Dominar o uso do algoritmo de agrupamento V.
- Dominar o processo de implementação de algoritmos de Machine Learning.

2 Implementação de código

2.1 Regressão linear

Defina aleatoriamente dez pontos de dados que apresentem uma relação linear entre si.

Etapa 1 – Importar pacotes dependentes.

Código:

```
# Importa o modelo de regressão linear
from sklearn.linear_model import LinearRegression
# A biblioteca de plotagem
import matplotlib.pyplot as plt
import numpy as np
```

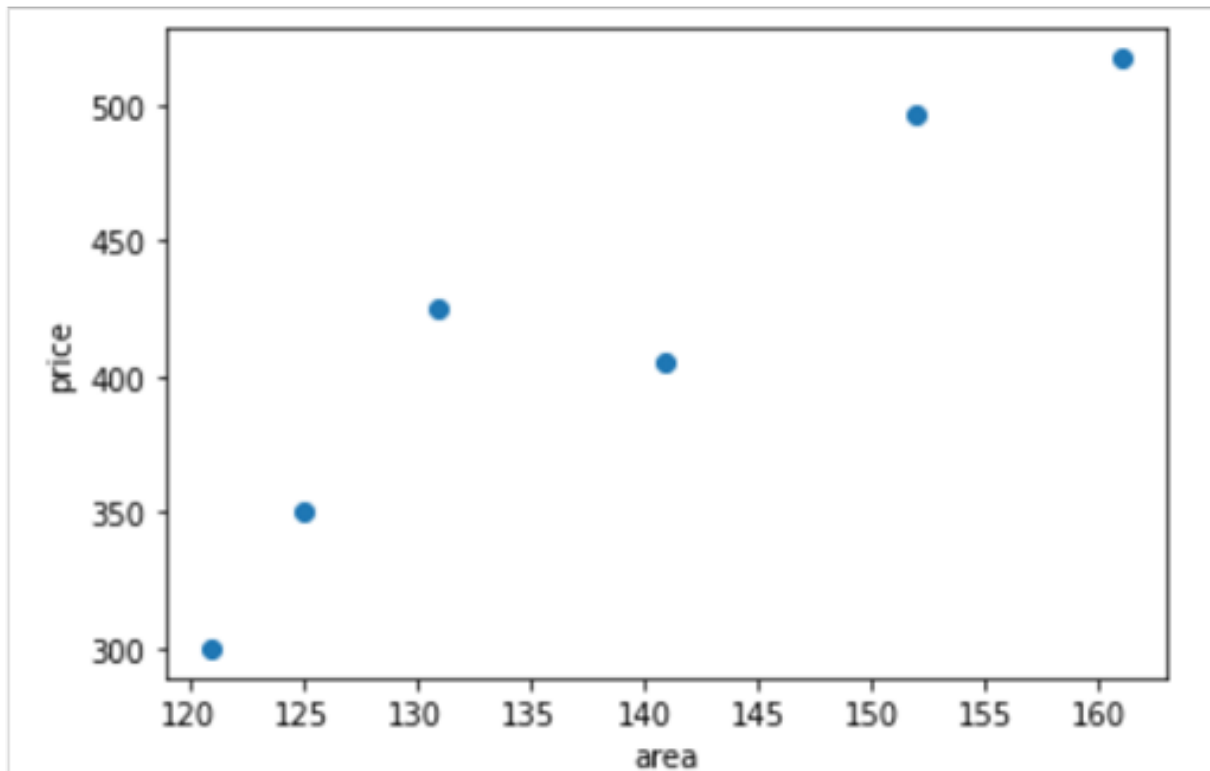
Etapa 2 – Criar e visualizar um conjunto de dados de preços de casas.

Código:

```
# x denota a área da casa como um atributo.
x = np.array([121, 125, 131, 141, 152, 161]).reshape(-1,1)
# y denota o preço da casa.
y = np.array([300, 350, 425, 405, 496, 517])
plt.scatter(x,y)
```

```
# Eixo X indica a área.  
plt.xlabel("area")  
# Eixo Y indica o preço.  
plt.ylabel("price")  
plt.show()
```

Saída:



Etapa 3 – Treinar o modelo.

Código:

```
# Encapsula o modelo de regressão linear em um objeto.  
lr = LinearRegression()  
# Treina o modelo com base no conjunto de dados.  
lr.fit(x,y)
```

Etapa 4 – Visualizar o modelo.

Código:

```
# Inclinação do modelo.  
w = lr.coef_  
# Intercepto do modelo.  
b = lr.intercept_  
print('Slope: ',w)  
print('Intercept: ',b)
```

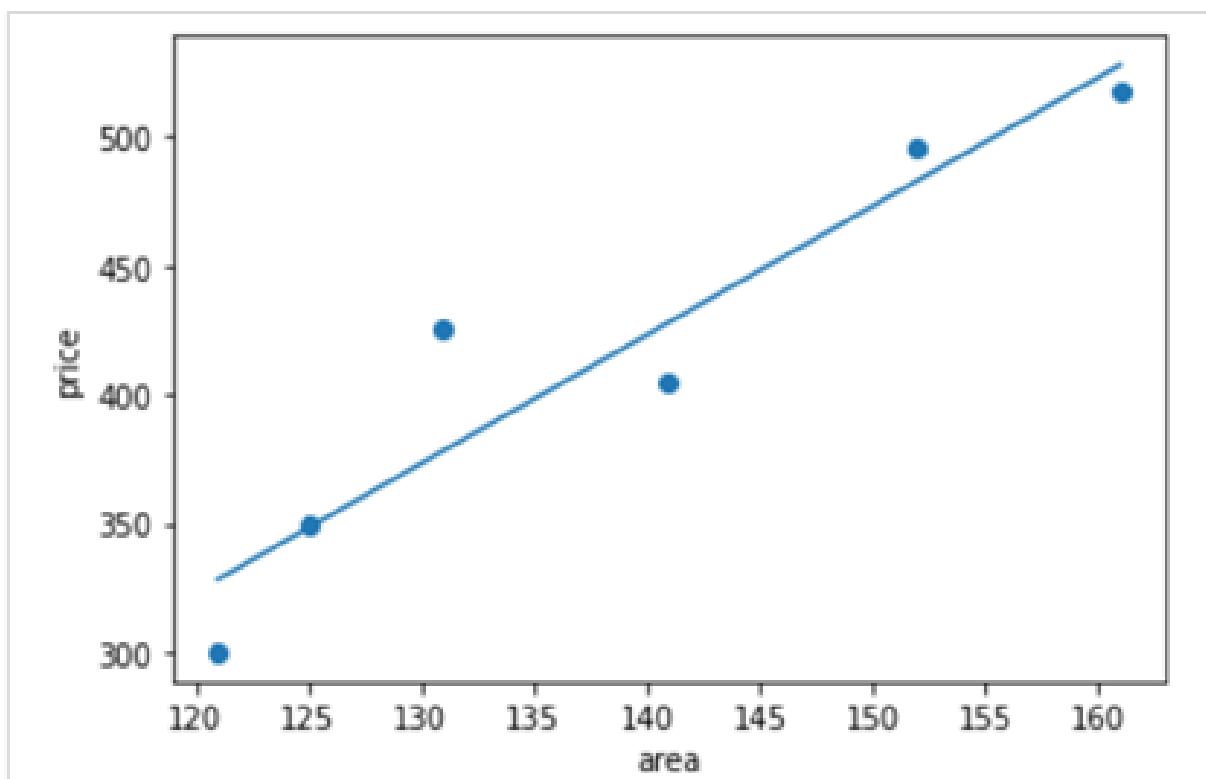
Saída:

Slope: [4.98467124]
Intercept: -274.8769665187576

Código:

```
plt.scatter(x,y)
plt.xlabel("area") # Eixo X indica a área.
plt.ylabel("price") # Eixo Y indica o preço.
plt.plot([x[0],x[-1]],[x[0]*w+b,x[-1]*w+b])
```

Saída:



Etapa 5 – Iniciar uma tarefa preditiva usando o modelo.

Código:

```
# Uma amostra de teste com uma área de 130.
testX = np.array([[130]])
lr.predict(testX)
Saída:
Array([373.13029447]) # O modelo prevê o preço da casa da amostra.
```

Saída:

```
# O modelo prevê o preço da casa da amostra.
Array([373.13029447])
```

2.2 Implementação de regressão linear (experimento de expansão)

Este experimento usa o conjunto de dados `lr2_data.txt`, que contém dados da área e dos preços de casas. Para obter o conjunto de dados, confira a Configuração de Ambiente do Laboratório.

Etapa 1 – Importar dependências.

Código:

```
import numpy as np
import matplotlib.pyplot as plt
```

Etapa 2 – Definir a função para calcular os gradientes.

Código:

```
def generate_gradient(X, theta, y):
    sample_count = X.shape[0]
    # Calcula o gradiente com base na matriz  $1/m \sum ((h(x^i) - y^i)) x_j^i$ 
    return (1./sample_count)*X.T.dot(X.dot(theta)-y)
```

Etapa 3 – Definir a função para ler os conjuntos de dados.

Código:

```
def get_training_data(file_path):
    # Ignora o título na primeira linha do conjunto de dados.
    orig_data = np.loadtxt(file_path, skiprows=1)
    cols = orig_data.shape[1]
    return (orig_data, orig_data[:, :cols - 1], orig_data[:, cols-1:])
```

Etapa 4 – Definir a função para inicializar os parâmetros.

Código:

```
# Inicializa o array theta.
def init_theta(feature_count):
    return np.ones(feature_count).reshape(feature_count, 1)
```

Etapa 5 – Definir a função para implementar a descida de gradiente.

Código:

```
def gradient_descending(X, y, theta, alpha):
    # Registra a tendência de mudança da função de custo  $J(\theta)$  para
    # confirmar que a descida de gradiente está correta.
    Jthetas = []
    # Calcula a função de perda, que é igual ao quadrado da diferença
    # entre o valor real e o valor previsto:  $(y^i - h(x^i))^2$ 
    Jtheta = (X.dot(theta)-y).T.dot(X.dot(theta)-y)
    index = 0
    # Calcula o gradiente.
    gradient = generate_gradient(X, theta, y)
    # Finaliza o cálculo quando o gradiente é inferior a 0,00001.
    while not np.all(np.absolute(gradient) <= 1e-5):
        theta = theta - alpha * gradient
```

```
# Calcula o novo gradiente.
gradient = generate_gradient(X, theta, y)
# Calcula a função de perda, que é igual ao quadrado da diferença
# entre o valor real e o valor previsto:  $(y^i - h(x^i))^2$ 
Jtheta = (X.dot(theta)-y).T.dot(X.dot(theta)-y)
# Registra o resultado a cada 10 cálculos.
if (index+1) % 10 == 0:
    Jthetas.append((index, Jtheta[0]))
index += 1
return theta, Jthetas
```

Etapa 6 – Definir a função para visualizar a curva de mudança da função de perda.

Código:

```
# Plota a curva de mudança da função de perda.
def showJTheta(diff_value):
    p_x = []
    p_y = []
    for (index, sum) in diff_value:
        p_x.append(index)
        p_y.append(sum)
    plt.plot(p_x, p_y, color='b')
    plt.xlabel('steps')
    plt.ylabel('loss function')
    plt.title('step - loss function curve')
    plt.show()
```

Etapa 7 – Definir a função para visualizar os pontos dos dados e a curva ajustada.

Código:

```
# Plota os pontos dos dados reais e a curva ajustada.
def showlinercurve(theta, sample_training_set):
    x, y = sample_training_set[:, 1], sample_training_set[:, 2]
    z = theta[0] + theta[1] * x
    plt.scatter(x, y, color='b', marker='x', label="sample data")
    plt.plot(x, z, 'r', color="r", label="regression curve")
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('liner regression curve')
    plt.legend()
plt.show()
```

Etapa 8 – Plotar os resultados.

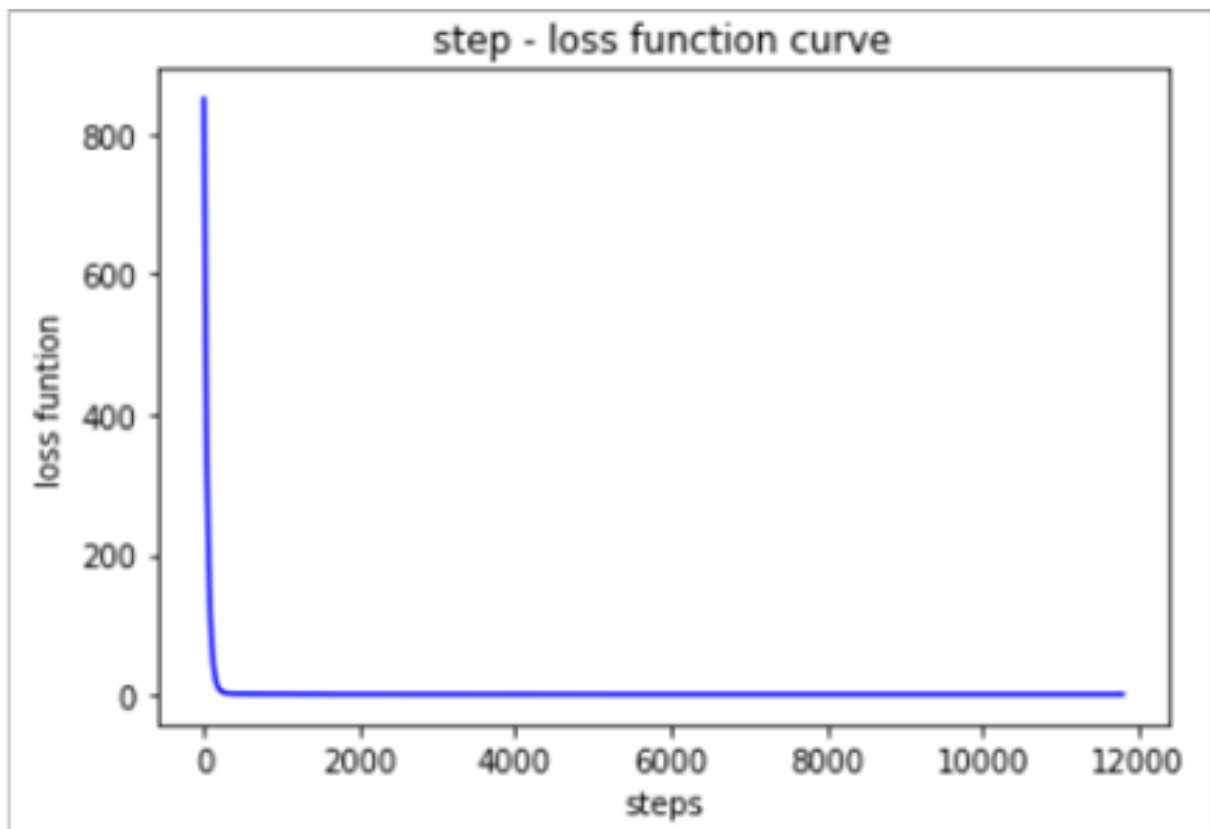
Código:

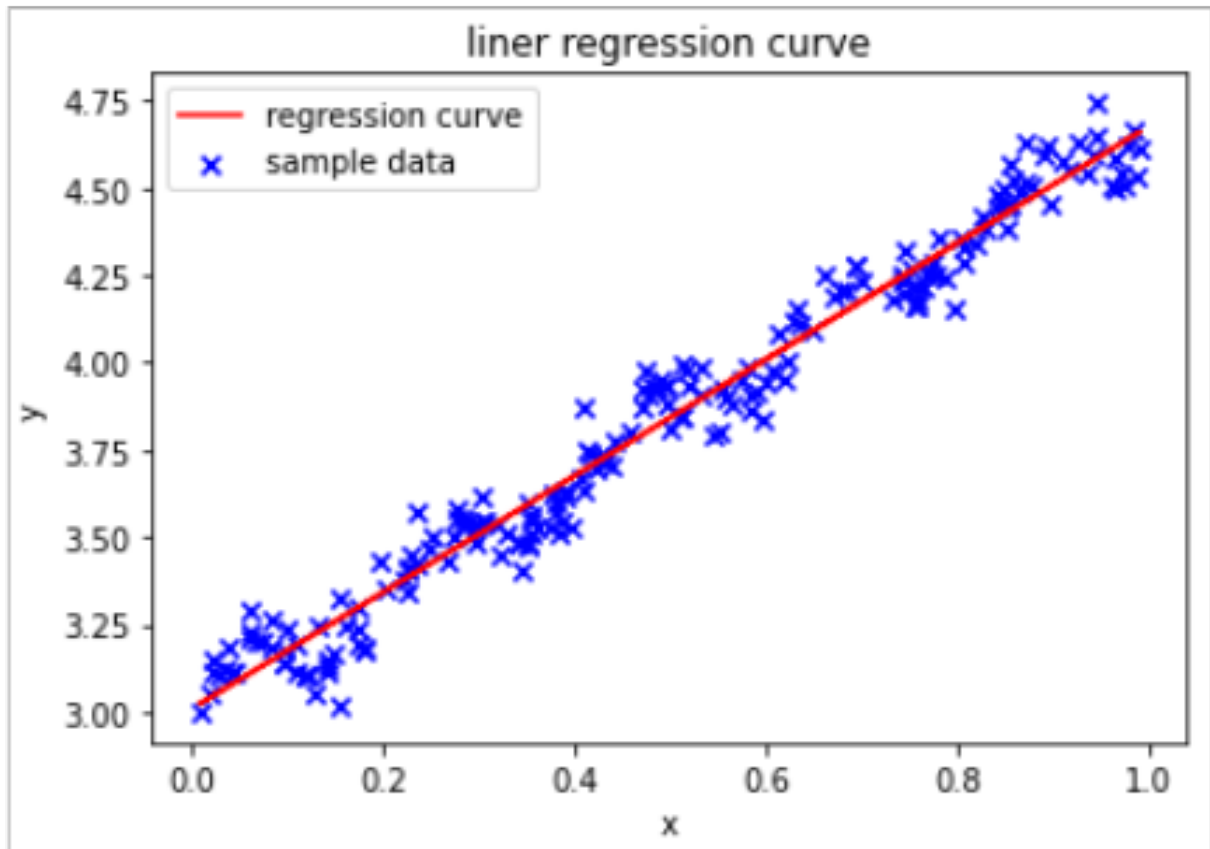
```
# Lê o conjunto de dados.
training_data_include_y, training_x, y = get_training_data("./ML/02/lr2_data.txt")
# Obtém o número de amostras e de atributos, respectivamente.
sample_count, feature_count = training_x.shape
# Define o passo de aprendizado alfa.
```

```
alpha = 0.01
# Inicializa theta.
theta = init_theta(feature_count)
# Obtém o parâmetro theta e o custo finais.
result_theta, Jthetas = gradient_descending(training_x, y, theta, alpha)
# Exibe o parâmetro.
print("w:{}".format(result_theta[0][0]), "b:{}".format(result_theta[1][0]))
showJTheta(Jthetas)
showlinercurve(result_theta, training_data_include_y)
```

Saída (ignore avisos):

```
w:3.0076279423997594 b:1.668677412281192
```





2.3 Regressão logística

Este experimento usa um conjunto de dados personalizado de aluguéis e áreas de casas. O conjunto de dados é definido durante a fase inicial do experimento.

Etapa 1 – Importar dependências.

Código:

```
# Importa o StandardScaler de sklearn.preprocessing.
from sklearn.preprocessing import StandardScaler
# Importa LogisticRegression de sklearn.linear_model.
from sklearn.linear_model import LogisticRegression
```

Etapa 2 – Definir o conjunto de dados.

Código:

```
# Cada item em X denota o aluguel e a área.
# y indica se o quarto deve ser alugado (0: não; 1: sim).
X=[[2200,15],[2750,20],[5000,40],[4000,20],[3300,20],[2000,10],[2500,12],[12000,80],
   [2880,10],[2300,15],[1500,10],[3000,8],[2000,14],[2000,10],[2150,8],[3400,20],
   [5000,20],[4000,10],[3300,15],[2000,12],[2500,14],[10000,100],[3150,10],
   [2950,15],[1500,5],[3000,18],[8000,12],[2220,14],[6000,100],[3050,10]
  ]
y=[1,1,0,0,1,1,1,1,0,1,1,0,1,1,0,1,0,0,0,1,1,1,0,1,0,1,0,1,0,1,1,0]
```

Etapa 3 – Pré-processar os dados. Padronize os dados para assegurar que a variância dos dados do atributo em cada dimensão seja 1 e que a média seja 0. Dessa forma, o resultado previsto não será dominado por grandes valores de atributos em algumas dimensões.

Código:

```
ss = StandardScaler()
X_train = ss.fit_transform(X)
Exiba os dados padronizados.
Entrada:
print(X_train)
```

Saída:

```
[[-0.60583897 -0.29313058]
 [-0.37682768 -0.09050576]
 [ 0.56003671 0.71999355]
 [ 0.14365254 -0.09050576]
 [-0.14781638 -0.09050576]
 [-0.68911581 -0.49575541]
 [-0.48092372 -0.41470548]
 [ 3.47472592 2.34099218]
 [-0.32269773 -0.49575541]
 [-0.56420055 -0.29313058]
 [-0.89730789 -0.49575541]
 [-0.27273163 -0.57680534]
 [-0.68911581 -0.33365555]
 [-0.68911581 -0.49575541]
 [-0.62665818 -0.57680534]
 [-0.10617796 -0.09050576]
 [ 0.56003671 -0.09050576]
 [ 0.14365254 -0.49575541]
 [-0.14781638 -0.29313058]
 [-0.68911581 -0.41470548]
 [-0.48092372 -0.33365555]
 [ 2.64195758 3.15149149]
 [-0.21027401 -0.49575541]
 [-0.29355084 -0.29313058]
 [-0.89730789 -0.69838024]
 [-0.27273163 -0.17155569]
 [ 1.80918923 -0.41470548]
 [-0.59751129 -0.33365555]
 [ 0.97642089 3.15149149]
 [-0.25191242 -0.49575541]]
```

Etapa 4 – Ajustar os dados.

Código:

```
# Usa o método fit de LogisticRegression para treinar os parâmetros do modelo.
lr = LogisticRegression()
lr.fit(X_train, y)
```


Saída:
LogisticRegression()

Prever os dados.

Código:

```
testX = [[2000,8]]
X_test = ss.transform(testX)
print("Value to be predicted: ",X_test)
label = lr.predict(X_test)
print("predicted label = ", label)
# Retorna a probabilidade prevista.
prob = lr.predict_proba(X_test)
print("probability = ",prob)
Saída:
Value to be predicted: [[-0.68911581 -0.57680534]]
predicted label = [1]
probability = [[0.41886952 0.58113048]]
```

2.4 Árvore de decisão

Este experimento usa o conjunto de dados tennis.txt, que contém 14 amostras. Cada amostra contém atributos relacionados ao tempo e se o tempo é propício para a prática de tênis.

Etapa 1 – Importar dependências.

Código:

```
import pandas as pd
import numpy as np
from sklearn import tree
import pydotplus
```

Etapa 2 – Definir a função para gerar a árvore de decisão.

Código:

```
# Gera a árvore de decisão.
def createTree(trainingData):
    data = trainingData.iloc[:, :-1] # Matriz dos atributos.
    labels = trainingData.iloc[:, -1] # Rótulos.
    # Classificador da árvore de decisão.
    trainedTree = tree.DecisionTreeClassifier(criterion="entropy")
    trainedTree.fit(data, labels) # Treina o modelo.
    return trainedTree
```

Etapa 3 – Definir a função para salvar o diagrama da árvore gerada.

Código:

```
def showtree2pdf(trainedTree,finename):
    # Exporta a árvore no formato Graphviz.
    dot_data = tree.export_graphviz(trainedTree, out_file=None)
```

```
graph = pydotplus.graph_from_dot_data(dot_data)
# Salva o diagrama da árvore na máquina local em formato PDF.
graph.write_pdf(finename)
```

Etapa 4 – Definir a função para gerar os dados vetorizados. Na função, `pd.Categorical(list).codes` obtém a lista de números de sequência correspondentes aos dados originais para converter as informações categóricas em informações numéricas.

Código:

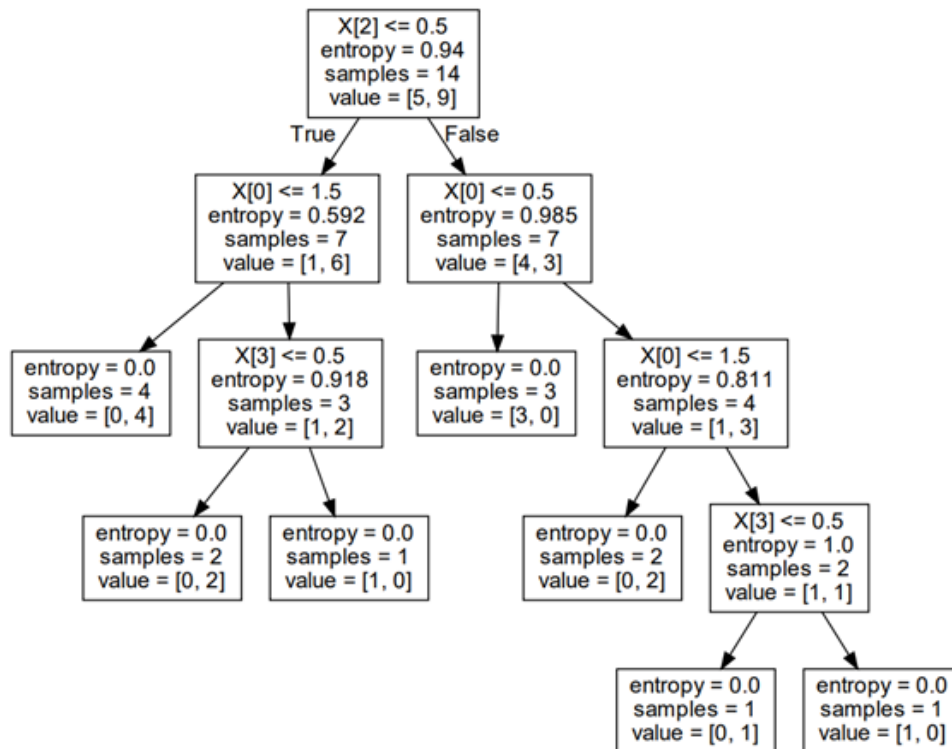
```
def data2vectoc(data):
    names = data.columns[:-1]
    for i in names:
        col = pd.Categorical(data[i])
        data[i] = col.codes
    return data
```

Etapa 5 – Invocar a função preditiva.

Código:

```
# Lê os dados de treinamento.
data = pd.read_table("tennis.txt", header=None, sep='\t')
# Vetoriza os dados.
trainingvec = data2vectoc(data)
# Cria a árvore de decisão.
decisionTree = createTree(trainingvec)
# Plota a árvore de decisão.
showtree2pdf(decisionTree, "tennis.pdf")
```

Um diagrama de árvore de decisão chamado `tennis.pdf` será gerado na máquina local.



O conteúdo do arquivo é uma exibição visual da árvore de decisão. No diagrama, $X[2]$ é a terceira variável de atributos (umidade); $X[0]$ é a primeira variável de atributos (tempo); $X[3]$ é a quarta variável de atributos (vento); entropy é o valor de entropia do nó; e samples é o número de amostras no nó, por exemplo, 14 no primeiro nó (nó raiz) indica o número de amostras no conjunto de treinamento; e value indica o número de amostras de diferentes tipos, por exemplo, no nó raiz, 5 indica o número de amostras de “não”, e 9 indica o número de amostras de “sim”.

Prever uma nova amostra.

Código:

```
# 0 tempo está ensolarado, a temperatura está baixa, a umidade está alta,
# e o vento está forte.
testVec = [0,0,1,1]
# Prevê o resultado.
print(decisionTree.predict(np.array(testVec).reshape(1,-1)))
```

Saída:

```
['No']
```

Etapa 5 (alternativa) – Invocar a função preditiva (sem Graphviz).

Código:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

```
# Lê os dados
data = pd.read_table("tennis.txt", header=None, sep='\t')

# Separa X e y
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# =====
# Codificação das features
# =====
X_encoded = []
encoders = []

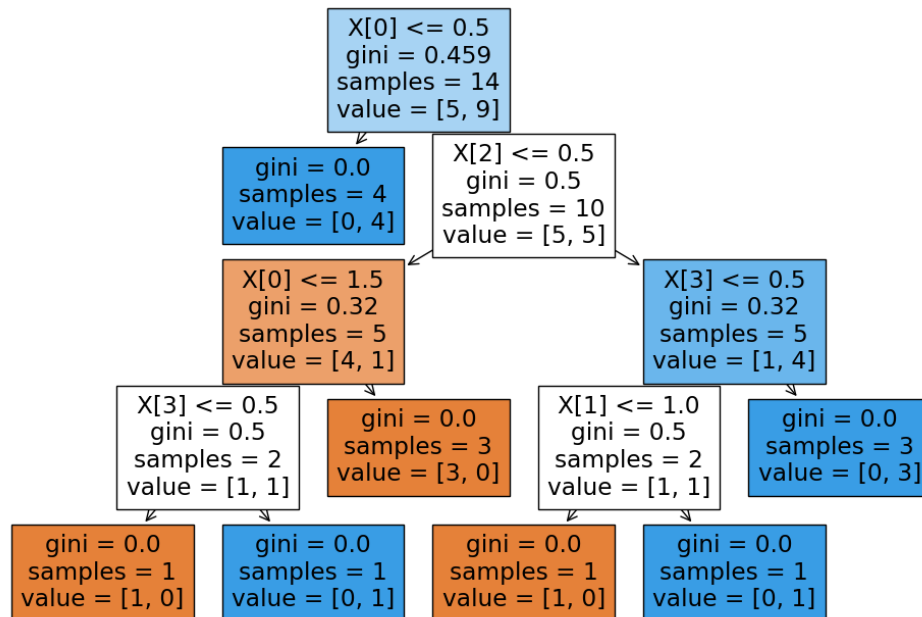
for col in X.T: # percorre cada coluna
    le = LabelEncoder()
    X_encoded.append(le.fit_transform(col))
    encoders.append(le)

# Transpor de volta
X_encoded = list(zip(*X_encoded))

# =====
# Codificação da saída
# =====
le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

# =====
# Treinamento (com ENTROPY)
# clf = DecisionTreeClassifier(criterion='entropy')
# =====
# Treinamento (com Gini)
clf = DecisionTreeClassifier()
# =====
clf.fit(X_encoded, y_encoded)
# =====
# Plot (sem Graphviz)
# =====
plt.figure(figsize=(12,8))
plot_tree(clf, filled=True)
plt.show()
```

Um diagrama de árvore de decisão será gerado tal como mostrado a seguir.
Saída:



O diagrama representa uma árvore de decisão construída para classificar se deve ou não jogar tênis com base em atributos como tempo ($X[0]$), temperatura ($X[1]$), umidade ($X[2]$) e vento ($X[3]$). A árvore inicia no nó raiz, que contém todas as amostras e realiza a primeira divisão com base no atributo mais relevante (tempo), separando os dados em subconjuntos mais homogêneos. Em cada nó, o valor de gini indica o grau de impureza (sendo zero quando todas as amostras pertencem à mesma classe), samples representa a quantidade de exemplos naquele ponto, e value mostra a distribuição entre as classes “No” e “Yes”. À medida que se desce na árvore, novas divisões são feitas com base em outros atributos, até que se atinjam nós folha, onde a decisão é final. As cores indicam a classe predominante em cada nó e a intensidade reflete o grau de pureza, permitindo visualizar claramente como o modelo aprende regras de decisão a partir dos dados.

Prever uma nova amostra.

Código:

```
# Amostra de teste correta (valores existentes no dataset)
testVec = ['sunny', 'cold', 'high', 'strong']

# Codifica usando os mesmos encoders
testVec_encoded = []
for i, val in enumerate(testVec):
    testVec_encoded.append(encoders[i].transform([val])[0])

# Predição
pred = clf.predict(np.array(testVec_encoded).reshape(1, -1))

# Converte para rótulo original
print(le_y.inverse_transform(pred))
```

Saída:

```
[ 'N' ]
```

2.5 Implementação de algoritmo K-means

Etapa 1 – Importar dependências. `make_blobs` é usado para gerar os conjuntos de dados necessários para esse experimento, `matplotlib` é usado para a visualização de dados, e `KMeans` é usado para ajustar os dados com base no algoritmo K-means.

Código:

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Etapa 2 – Gerar o conjunto de dados. `make_blobs()` é usado para gerar um conjunto de dados a uma tarefa de agrupamento retornando o conjunto de dados gerado e os rótulos. Observação: o algoritmo K-means é geralmente usado para processar conjuntos de dados não rotulados em produção. O conjunto de dados gerado contém rótulos apenas para facilitar o entendimento.

Código:

```
X, y = make_blobs(n_samples=500, n_features=2, centers=4, random_state=1)
```

`n_samples` é o número de amostras, `n_features` é o número de atributos de cada amostra (a dimensão dos dados), `centers` é o número de categorias, e `random_state` é a semente para o gerador de números aleatórios. A semente assegura que os dados gerados sejam os mesmos a cada vez.

Exiba as informações de dimensão do conjunto de dados gerado.

Código:

```
Print("Dimension of X is {}".format(X.shape))
Print("Dimension of y is {}".format(y.shape))
```

Saída:

```
Dimension of X is (500, 2)
Dimension of y is (500,).
```

O conjunto de dados contém 500 pontos de dados, e cada ponto de amostra contém dois atributos.

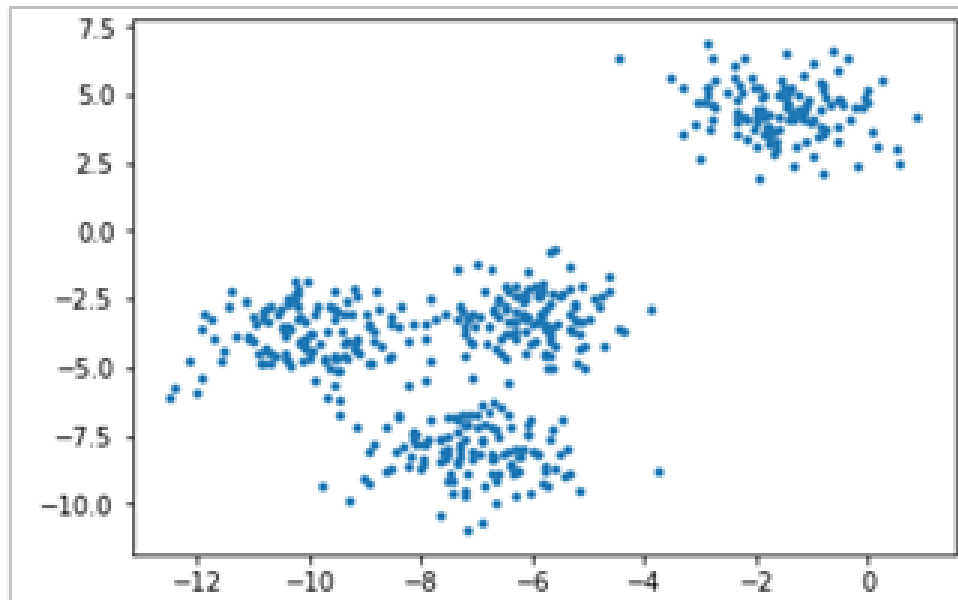
Etapa 3 – Desenhar gráficos de dispersão. Primeiro, desenhe um gráfico de dispersão do conjunto de dados diretamente. Os rótulos gerados durante a geração do conjunto de dados não serão considerados no momento. O gráfico contém quatro agrupamentos diferentes.

Código:

```
fig, ax1 = plt.subplots(1)
ax1.scatter(X[:, 0], X[:, 1],
            marker='o' # Define o formato do ponto como um círculo.
            ,s=8 # Define o tamanho do ponto.
```

```
)  
plt.show()
```

Saída:

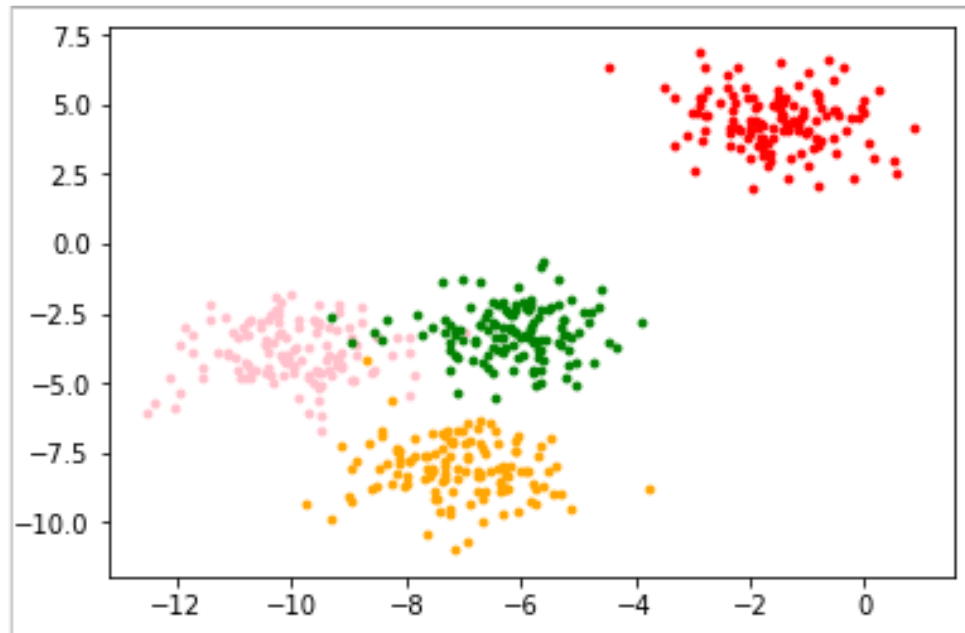


Em seguida, desenhe um gráfico de dispersão no qual pontos de diferentes cores correspondem aos rótulos gerados.

Código:

```
color = ["red","pink","orange","green"]  
fig, ax1 = plt.subplots(1)  
  
for i in range(4):  
    ax1.scatter(X[y==i, 0], X[y==i, 1] # Desenha a cor com base no rótulo.  
               ,marker='o' # Define o formato do ponto como um círculo.  
               ,s=8 # Define o tamanho do ponto.  
               ,c=color[i]  
    )  
plt.show()
```

Saída:



Etapa 4 – Realizar o agrupamento k-means. Use o `sklearn.cluster.KMeans` fornecido pelo `scikit-learn` para implementar o agrupamento K-means. Primeiro, agregue as amostras de dados em três tipos.

Código:

```
n_clusters = 3
cluster1 = KMeans(n_clusters=n_clusters,random_state=0).fit(X)
```

Veja o rótulo de cada ponto de amostra após o agrupamento.

Código:

```
y_pred1 = cluster1.labels_
print(y_pred1)
```

Saída:

```
[0 0 2 1 2 1 2 2 2 2 0 0 2 1 2 0 2 0 1 2 2 2 2 1 2 2 1 1 2 2 0 1 2 0 2 0 2
 2 0 2 2 2 1 2 2 0 2 2 1 1 1 2 2 2 0 2 2 2 2 2 1 1 2 2 1 2 0 2 2 2 0 2 2 0
 2 2 0 2 2 2 1 1 2 1 1 2 2 1 2 2 1 0 2 2 1 0 0 2 0 1 1 0 1 2 1 2 2 1 1 2 2
 0 1 2 1 2 1 2 1 2 2 0 0 2 2 2 1 0 0 2 1 2 2 2 2 0 1 2 1 1 2 0 2 1 1 1 2 2
 0 0 2 2 1 0 1 2 2 2 2 2 2 2 2 1 0 0 0 2 1 0 2 2 0 1 2 2 2 2 0 2 2 1 0 0
 2 2 0 0 2 1 1 0 0 2 1 2 0 0 1 0 2 1 2 2 0 2 2 0 2 2 2 2 0 2 2 2 1 2 1 2 0
 2 2 2 2 2 1 2 1 0 2 0 2 1 1 2 0 1 0 2 2 0 0 0 2 2 0 2 2 1 1 2 2 1 2 2 2
 1 2 1 2 2 1 2 0 0 2 2 2 2 1 1 2 1 2 0 1 0 1 0 0 1 0 1 1 2 2 2 2 2 2 0 1
 0 0 0 2 2 2 0 2 0 0 2 0 0 2 1 0 2 2 1 1 2 0 1 1 2 0 1 1 2 2 1 2 2 0 0 1 2
 0 2 1 1 2 2 2 0 2 1 1 2 1 1 1 0 0 2 1 2 2 0 1 2 1 2 1 2 2 2 1 2 2 0 1 0
 0 0 0 0 0 2 0 1 0 1 1 2 1 2 2 2 0 1 2 1 2 0 2 2 0 2 2 1 1 0 2 2 1 2 2 0 0
 2 0 2 2 0 2 0 2 1 0 1 2 2 1 2 2 1 0 2 1 1 2 2 2 2 0 1 0 2 1 0 0 0 2 1 2 0
 2 2 2 2 0 2 2 2 2 2 2 0 2 2 0 2 1 2 1 2 2 2 1 1 1 2 2 2 0 2 1 2 0 1 0 1 0
 2 1 1 0 2 2 0 2 2 2 0 2 1 2 2 0 0 0 2]
```

Imprima o centroide de cada agrupamento.

Código:

```
centroid1 = cluster1.cluster_centers_  
print centroid1
```

Saída:

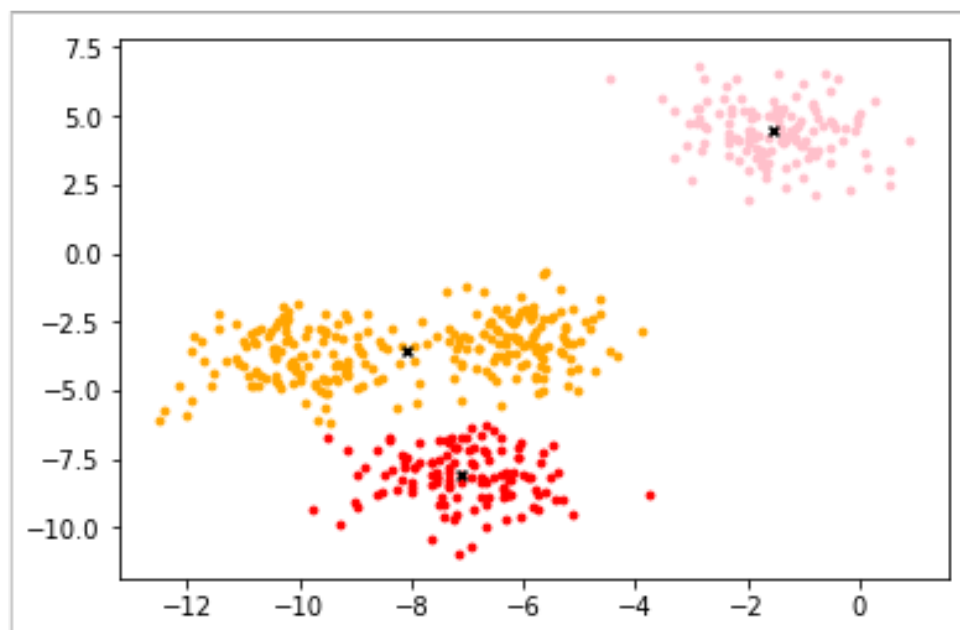
```
[[ -7.09306648 -8.10994454]  
 [-1.54234022  4.43517599]  
 [-8.0862351  -3.5179868  ]]
```

Visualize o resultado do agrupamento.

Código:

```
color = ["red","pink","orange","gray"]  
  
fig, ax1 = plt.subplots(1)  
  
for i in range(n_clusters):  
    ax1.scatter(X[y_pred1==i, 0], X[y_pred1==i, 1]  
                ,marker='o' # Define o formato do ponto como um círculo.  
                ,s=8 # Define o tamanho do ponto.  
                ,c=color[i]  
                )  
  
ax1.scatter centroid1[:,0],centroid1[:,1]  
        ,marker="x"  
        ,s=15  
        ,c="black")  
plt.show()
```

Saída:



O gráfico mostra que as amostras de dados estão agregadas em três agrupamentos, e o centroide de cada agrupamento é representado por x.

Etapas 5 – Realizar o agrupamento k-means novamente. Em seguida, realize as etapas anteriores para usar o agrupamento k-means a fim de agregar as amostras de dados em quatro tipos.

Código:

```
n_clusters = 4
cluster2 = KMeans(n_clusters=n_clusters,random_state=0).fit(X)
y_pred2 = cluster2.labels_
centroid2 = cluster2.cluster_centers_
print("Centroid: {}".format(centroid2))
```

Saída:

```
Centroid: [[ -6.08459039 -3.17305983]
 [ -1.54234022  4.43517599]
 [ -7.09306648 -8.10994454]
 [-10.00969056 -3.84944007]]
```

Visualize o resultado do agrupamento.

Código:

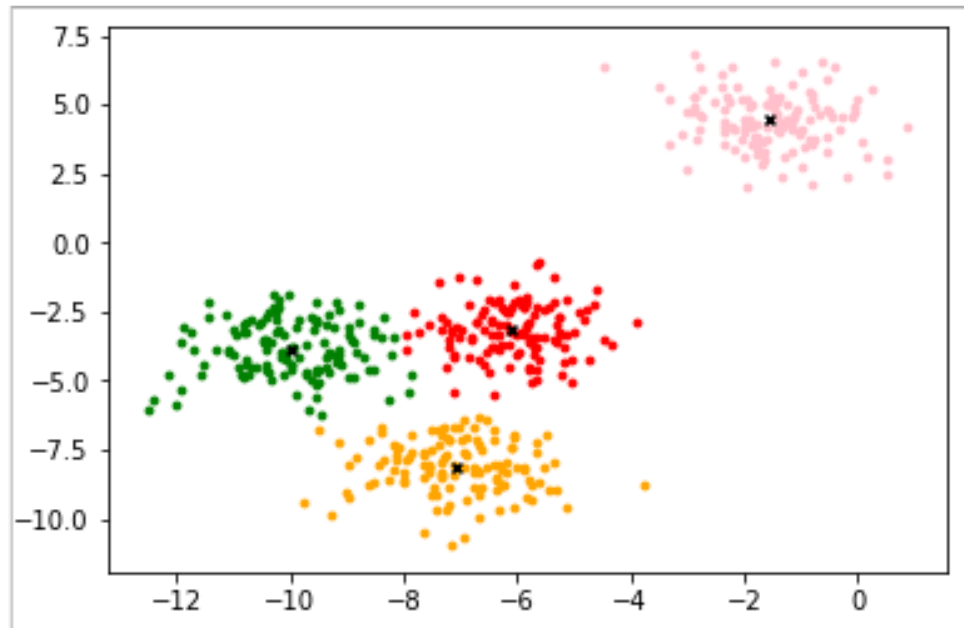
```
color = ["red","pink","orange","green"]

fig, ax1 = plt.subplots(1)

for i in range(n_clusters):
    ax1.scatter(X[y_pred2==i, 0], X[y_pred2==i, 1]
                ,marker='o' # Define o formato do ponto como um círculo.
                ,s=8 # Define o tamanho do ponto.
                ,c=color[i]
                )

ax1.scatter(centroid2[:,0],centroid2[:,1]
            ,marker="x"
            ,s=15
            ,c="black")
plt.show()
```

Saída:



O gráfico mostra que as amostras de dados estão agregadas em quatro agrupamentos, e o centroide de cada agrupamento é representado por x. Compare os gráficos de dispersão gerados com base nos dados originais e nos dados agregados em quatro tipos. É possível ver que muitos pontos de amostras estão agregados em agrupamentos errados.

3 Questões

1. Como implementar o k-NN do zero usando Python?

Implemente um classificador k-NN sem utilizar bibliotecas prontas de aprendizado de máquina (como *scikit-learn*). Sua implementação deve incluir as etapas de treinamento, cálculo de distância e predição.

Esqueleto do código:

```
import numpy as np
from collections import Counter

class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        """
        Armazena os dados de treinamento.
        """
        # TODO: armazenar X e y
        pass

    def _distance(self, x1, x2):
        """
```

```
Calcular a distância entre dois pontos.
Sugestão: usar distância Euclidiana.
"""

# To do: implementar distância
pass

def _predict(self, x):
    """
    Prediz a classe de um único exemplo.
    """
    # To do:
    # 1. Calcular a distância de x para todos os pontos de treino
    # 2. Selecionar os k vizinhos mais próximos
    # 3. Obter os rótulos desses vizinhos
    # 4. Retornar a classe mais frequente
    pass

def predict(self, X):
    """
    Prediz a classe para múltiplos exemplos.
    """
    # To do: aplicar _predict para cada elemento de X
    pass

# =====
# Exemplo de uso
# =====

X_train = [
    [1, 2],
    [2, 3],
    [3, 3],
    [6, 5],
    [7, 7],
    [8, 6]
]

y_train = ['A', 'A', 'A', 'B', 'B', 'B']

model = KNN(k=3)
model.fit(X_train, y_train)

X_test = [[5, 5]]

# To do: realizar a predição
# print(model.predict(X_test))
```



Observações:

- Utilize a distância Euclidiana.
- Em caso de empate na votação, escolha uma estratégia de desempate (por exemplo, o primeiro mais próximo).
- Teste sua implementação com diferentes valores de k .

4 Resumo

Este laboratório apresenta o processo de implementação de Machine Learning, incluindo a importação, a segmentação e a padronização de dados, bem como modelos e hiper-parâmetros. Algoritmos de Machine Learning comuns são implementados com base no `scikit-learn` para ajudar você a melhor entender como criar e usar modelos de Machine Learning.



Referências

1. HUAWEI TECHNOLOGIES CO., LTD. **Artificial Intelligence Technology and Application: Python Lab Guide**. Huawei Technologies Co., Ltd., China.
2. David Beazley and Guido Van Rossum. **Python: Essential Reference**. *New Riders Publishing*, USA, 1999.